

ЛАБОРАТОРНАЯ РАБОТА №8

Разработка сценариев на языке оболочки *bash*

1. Цель работы

1. Изучение принципов разработки сценариев на языке программирования оболочки *bash*.
2. Разработка собственного сценария с использованием основных синтаксических элементов и команд языка.
3. Знакомство с методикой отладки сценариев для поиска и устранения ошибок.

2. Теоретическая часть

Основы разработки сценариев оболочки

Взаимодействие пользователя с операционной системой осуществляется через оболочку, которая представляет собой внешнюю программу.

Сразу после запуска оболочки производится её инициализация для установки ряда параметров. При этом оболочкой производится чтение двух файлов: */etc/profile* и *~/.profile*. В первом из них содержатся настройки параметров, общие для всех пользователей. Во втором файле каждый пользователь может разместить свои собственные настройки для работы с оболочкой.

Пользовательская оболочка может быть запущена на выполнение в двух режимах – интерактивном и не интерактивном. Когда оболочка выдаёт пользователю приглашение, она работает в интерактивном режиме. Это означает, что оболочка принимает ввод от пользователя и выполняет команды, которые пользователь укажет. Оболочка называется интерактивной, поскольку она взаимодействует с пользователем. Завершение работы с оболочкой в этом случае происходит по команде пользователя.

В не интерактивном режиме оболочка не взаимодействует с пользователем. Вместо этого она читает команды из некоторого файла и выполняет их. Когда будет достигнут конец файла, оболочка завершится. Запуск оболочки в не интерактивном режиме можно осуществить следующим способом:

\$ /bin/sh имя_файла

Здесь *имя_файла* — файл, содержащего команды для выполнения. Такой файл называется сценарием оболочки. Он является текстовым файлом и может быть создан любым доступным текстовым редактором.

Пользователю может представлять неудобство каждый раз указывать имя программы-оболочки */bin/sh* для выполнения сценария. Для того чтобы иметь возможность выполнять сценарий, набирая только его имя, прежде всего необходимо сделать его исполняемым. Для этого необходимо установить соответствующие права доступа к файлу с помощью команды *chmod* следующим способом:

chmod u+x имя_файла

Кроме этого, необходимо явно указать, какая оболочка должна использоваться для выполнения данного сценария. Это можно сделать, разместив в первой строке сценария последовательность символов *#!/bin/sh*. Здесь указывается, что для выполнения сценария необходимо использовать оболочку */bin/sh*.

Сценарий может содержать комментарии. Комментарий — это оператор, который может размещаться в сценарии оболочки, но оболочкой не исполняется. Комментарий начинается с символа *#* и продолжается до конца строки.

Ниже приведён пример короткого сценария:

#!/bin/sh

#пример короткого сценария

date

who

Составные команды

Кроме простых команд можно формировать составные команды. Они представляют собой упорядоченный набор простых команд, связанных между собой при помощи специальных операторов управления (ОУ). Составная команда имеет следующий синтаксис:

команда ОУ команда ОУ ... ОУ команда

В таблице приведены наиболее часто используемые операторы управления и даны пояснения по выполнению соответствующих составных команд.

Оператор управления в составной команде	Правило выполнения
<i>команда1 ; команда2</i>	«команда2» выполняется после завершения выполнения «команда1».
<i>команда1 && команда2</i>	«команда2» выполняется после завершения выполнения «команда1» если код завершения «команда1» 0 (истина).
<i>команда1 команда2</i>	«команда2» выполняется после завершения «команда1» если код завершения «команда1» отличен от нуля (ложь).

Переменные

Переменная это «слово», которому приписано значение. Оболочка разрешает создавать и удалять переменные, а также присваивать им значения. В большинстве случаев разработчик ответственен за управление переменными в сценариях. Использование переменных позволяет

создавать гибкие, легко адаптируемые сценарии. Определяются переменные следующим образом (отсутствие пробелов до и после символа «=» существенно):

имя_переменной—значение

Например, `MY_NAME=Sergey` определяет переменную с именем `MY_NAME` и присваивает ей значение `Sergey`. Имена переменных определяются по тем же правилам, что и в языке программирования C. В переменной можно сохранять любое нужное значение. Особый случай когда это значение содержит пробелы. Для правильного выполнения такого действия указанное значение достаточно заключить в кавычки.

Для того чтобы получить значение переменной, перед её именем необходимо поставить знак `$`. В том случае, когда некоторая переменная становится ненужной, её можно удалить операцией `unset имя_переменной`. Ниже приведён пример, иллюстрирующий работу с переменными в сценариях.

```
#!/bin/sh
#пример операций над переменными
MY_NAME=Sergey
MY_FULL_NAME="Sergey Borisovich Sidorov"
echo name = $MY_NAME and full name = $MY_FULL_NAME
unset MY_NAME
```

В результате выполнения команды `echo` на терминал будет выдано сообщение:
`name=Sergey and full name = Sergey Borisovich Sidorov`

Все рассмотренные выше примеры переменных — это примеры скалярных переменных, то есть таких, которые могут хранить только одно значение. Наряду с ними можно использовать *переменные-массивы*. Доступ к элементам массива осуществляется операцией индексирования `[]`. Язык программирования оболочки не требует предварительного объявления переменной массивом с указанием его размерности. При этом отдельные элементы массива создаются по мере доступа к ним. Ниже приведён пример работы с массивом `NAME`.

```
#!/bin/sh
#пример использования массива
NAME[0]=Sergey
NAME[1]=Katya
NAME[2]=Liza
echo все имена: ${NAME[*]}
echo ${NAME[1]} и ${NAME[2]} сестры
```

Если вместо индекса элемента использовать `*`, результатом выражения будет список значений всех элементов массива (в данном случае таковых три).

Область видимости переменных

Каждая переменная имеет свою *область видимости*. Допустим, в сценарии определена некоторая переменная. Что будет с ней после завершения этого сценария, доступна ли она из

других сценариев, вызываемых исходным? В языке оболочки все переменные делятся на три категории: локальные переменные, переменные окружения и переменные оболочки.

Локальная переменная – это такая переменная, которая существует только внутри конкретного экземпляра оболочки. Она не доступна программам, запускаемым на выполнение из этой оболочки. Все рассматривавшиеся ранее переменные были локальными.

Переменная окружения – это переменная, которая доступна любой программе, запущенной из данной оболочки. Некоторые программы нуждаются в определённых переменных окружения. В таком случае в сценарии их можно определить.

Переменная оболочки – это специальная переменная, которая устанавливается оболочкой и необходима ей для корректной работы. Некоторые из них являются переменными окружения, а некоторые – локальными. В переменных с именами 1, 2, 3 сохраняются параметры командной строки. То есть если некий сценарий *script* запустить в виде *script something*, то в его оболочке *\$1=something*.

Переменную можно разместить в окружении, выполнив команду экспорта: *export имя_переменной*. В результате выполнения следующего сценария на терминал будут выданы значения всех переменных окружения оболочки и, в том числе, переменной *MY_NAME*.

```
#!/bin/sh
# пример размещения переменной в окружении
MY_NAME=Sergey ; export MY_NAME;
set
```

Ниже в таблице приводится список некоторых стандартных переменных окружения и переменных оболочки с указанием их назначения и примера их инициализации

<i>Имя переменной</i>	<i>Краткое описание</i>
HOME	Содержит путь к домашнему каталогу пользователя
TERM	Тип используемого терминала <i>TERM=vt100</i>
PATH	Определяет список каталогов, в которых оболочка ищет запускаемые на выполнение программы <i>PATH=/bin:/usr/bin</i>
MANPATH	Определяет список каталогов, в которых программа <i>man</i> ищет справочную информацию по запрошенной команде <i>MANPATH=/usr/man:/usr/share/man</i>
PWD	Содержит путь к текущему рабочему каталогу
RANDOM	Формирует случайное целое число из диапазона 0..32767 каждый раз при доступе к этой переменной
?	Результат выполнения предыдущей команды
HOSTNAME	Имя узла (компьютера), на котором выполняется оболочка
SECONDS	Число секунд, прошедших с момента запуска оболочки

Средства ввода-вывода

Задачи ввода—вывода могут быть решены посредством использования специальных команд *echo*, *printf*, *read*. Команда *echo* выдаёт на стандартное устройство вывода значения всех своих параметров. Команда *printf* подобна своему аналогу функции *printf* в программах на языке С. Первым параметром указывается форматная строка, а далее перечисляются выдаваемые значения.

Команда *read имя1 имя2 ... имяN* считывает со стандартного устройства ввода строку, выделяет из неё отдельные слова (группы символов, отделяемые пробелами) и каждое слово заносит в указанные в качестве параметров соответствующие переменные. При этом если переменных меньше, чем выделенных слов, то в последнюю из них будет занесена оставшаяся часть строки.

В качестве примера приведён сценарий, в котором от пользователя запрашивается строка, после чего она выводится на терминал. Параметр *-n* запрещает перевод на следующую строку по окончании вывода командой *echo*, разрешенный по умолчанию.

```
#!/bin/sh
# пример использования средств ввода-вывода
echo -n Введите строку:
read INPUT
printf "%s\n" "$INPUT"
```

Подстановки

Выделяют следующие виды подстановок:

- подстановка имён файлов — позволяет выполнять преобразование строки, содержащей шаблоны, в список имён файлов;
- подстановка переменной — позволяет управлять значением переменной, основываясь на ее состоянии;
- подстановка команды — позволяет захватить вывод команды и подставить его в другую команду
- арифметическая подстановка — позволяет выполнять простые математические вычисления, используя оболочку.

Подстановка имён файлов основывается на использовании различных шаблонов для задания группы файлов в кратком виде вместо полного их перечисления. Шаблон *** отождествляется с любым количеством любых символов в имени файла. Шаблон *?* соответствует одному произвольному символу, а шаблон *[группа символов]* явно определяет набор символов, допустимых в месте его расположения. Ниже приведены примеры использования шаблонов:

<i>*.cpp</i>	<i>#список файлов, оканчивающихся на .cpp</i>
<i>module?.h</i>	<i>#список файлов с одним произвольным символом вместо ?</i>
<i>module[aA].cpp</i>	<i>#modulea.cpp и ИЛИ moduleA.cpp</i>

Механизм подстановки команды состоит в выполнении оболочкой указанного набора команд и последующей подстановки их вывода вместо самих команд. Подстановка команды выполняется при записи команды в обратных апострофах. Например, в результате выполнения команды *DATE= 'date'* переменная *DATE* примет значение – вывод команды *date*.

Для вычисления значения арифметического выражения *expression* необходимо использовать команду следующего вида: *\$((expression))* При этом в выражении могут использоваться операции сложения, вычитания, умножения и целочисленного деления, а также круглые скобки для задания порядка вычислений. Например, в результате выполнения следующей команды переменная *foo* примет значение 3.

```
foo=$((5+3*2)-4)/2))
```

Команды ветвления

Язык оболочки содержит два оператора ветвления: *if* и *case*.

Оператор *if* выполняет действия в зависимости от истинности заданного условия и имеет следующий синтаксис:

```
if list1
then
    commands1
elif list2
then
    commands2
else
    commands3
fi
```

В приведённой записи как *elif*, так и *else* могут отсутствовать. Если тело оператора *if* небольшое, то обычно используют запись в одну строку такую как: *if list1 ; then commands1 ; else commands3 fi*; При этом важно правильно ставить *;*. Алгоритм выполнения оператора *if* следующий:

1. выполняется *list1*
2. если код завершения *list1* 0 (истина), то выполняется *commands1* и оператор *if* завершается;
3. иначе выполняется *list2* и проверяется его код завершения;
4. если *list2* возвратил 0, то выполняется *commands2* и оператор завершается;
5. если *list2* возвратил не нулевое значение, то выполняется *commands3*.

В качестве условий *list1* и *list2* могут использоваться обычные команды, однако наиболее часто – это вызов одной или нескольких *test* команд в виде *test выражение* или (более кратко) *[выражение]*. Подробно команда *test* рассмотрена в следующем разделе. Ниже приведён пример использования оператора *if*:

```
#!/bin/sh
# выполнение программы с контролем её существования
if [ -x $HOME/script ] ; then $HOME/script ; fi ;
```

В данном примере проверяется, существует ли в домашнем каталоге пользователя файл с именем *script* и является ли он исполняемым (атрибут 'x'). Если это так, то сценарий запускает *script* на выполнение.

Если необходимо сделать выбор из многих альтернатив, то более удобно использовать оператор *case*, имеющий следующий синтаксис:

```
case значение in
  шаблон1) список1 ;;
  шаблон2) список2 ;;
  ...
esac
```

Оператор *case* является аналогом оператора *switch* в языке C и работает похожим образом. Строка *значение* сравнивается с каждым из указанных *шаблонов* до тех пор, пока не будет обнаружено соответствие. После обнаружения соответствия выполняется *список*, указанный после шаблона. Два символа ;; после списка имеют тот же смысл, что и оператор *break* в программах на языке C. Если никаких соответствий не обнаружено, то оператор *case* завершается без выполнения каких-либо действий.

Ниже приведён пример использования оператора *case*. Сценарий ожидает в качестве параметра название фрукта и выдаёт его описание. Если при вызове параметр не указан, то выдаётся диагностическое сообщение.

```
#!/bin/sh
# печать информации о фруктах
if [ -z "$1" ]
then
    echo "Ошибка: пропущен параметр"
    exit 1
fi
case "$1" in
    apple) echo "Яблоки очень полезны" ;;
    banana) echo "Бананы очень вкусны" ;;
    *) echo "Ничего не могу сказать про" ;1 ;;
esac
```

Синтаксис команды test

Команда *test выражение* проверяет указанное *выражение* и заканчивает свою работу с кодом завершения 0 (истина) или (ложь). В качестве выражение могут использоваться выражения 3 типов:

- проверки характеристик файлов;
- сравнения строк;
- числовые сравнения.

Основной синтаксис для проверки характеристик файлов следующий: *test опция имя_файла* В приведённой ниже таблице сведены возможные опции команды *test* для проверки характеристик файлов.

Опция	Описание
<i>-b</i>	Истина, если файл существует и является блочным специальным файлом.
<i>-c</i>	Истина, если файл существует и является символьным специальным файлом.
<i>-d</i>	Истина, если файл существует и является каталогом.
<i>-e</i>	Истина, если файл существует.
<i>-f</i>	Истина, если файл существует и является обычным файлом.
<i>-h</i>	Истина, если файл существует и является символической ссылкой.
<i>-r</i>	Истина, если файл существует и доступен на чтение.
<i>-s</i>	Истина, если файл существует и имеет ненулевой размер.
<i>-S</i>	Истина, если файл существует и является сокетом.
<i>-w</i>	Истина, если файл существует и доступен по записи.
<i>-x</i>	Истина, если файл существует и доступен на выполнение.

В приведённой ниже таблице сведены возможные способы указания использования команды *test* для проверки строк.

Опция	Описание
<i>-z строка</i>	Истина, если строка имеет нулевую длину.
<i>-n строка</i>	Истина, если строка имеет ненулевую длину.
<i>строка1 = строка2</i>	Истина, если строки совпадают.
<i>строка1 != строка2</i>	Истина, если строки различны.

При использовании числовых сравнений команда *test* имеет следующий синтаксис: *test число1 оператор число2* В приведённой ниже таблице сведены возможные значения операторов при числовых сравнениях.

Опция	Описание
<i>-eq</i>	Истина, если числа равны.
<i>-ne</i>	Истина, если числа не равны.
<i>-lt</i>	Истина, если меньше.

<i>-le</i>	Истина, если меньше или равны.
<i>-gt</i>	Истина, если больше.
<i>-ge</i>	Истина, если больше или равны.

Организация циклов

Язык оболочки позволяет организовывать циклическое выполнение команд. В распоряжение пользователю предлагается несколько вариантов циклов:

- *while*
- *for*
- *select*

Оператор цикла *while* имеет следующий синтаксис:

while команда

do

список

done

При выполнении цикла сначала выполняется *команда*. Если результат ненулевой, то происходит выход из цикла, в противном случае выполняется тело цикла *список* и происходит переход на следующую итерацию. Хотя в качестве условия *команда* может использоваться любая допустимая в GNU/Linux команда, обычно это команда *test*. Ниже приведён пример выдачи на терминал десяти последовательных чисел от 0 до 9.

```
#!/bin/sh
# арифметические вычисления
x=0
while [ $x -lt 10 ] # значение переменной x меньше 10 ?
do
    echo $x
    x='expr $x + 1' # увеличение x на 1
done
```

Цикл *for* выполняет команды из *список* для каждого элемента из списка и имеет следующий синтаксис:

```
for имя in элемент1 элемент2 ... элементN
do
```

```
        список
done
```

В качестве элементов можно использовать различные шаблоны. В приведённом ниже примере, все файлы из домашнего каталога пользователя, которые заканчиваются на *.bash*, копируются в каталог *scripts* и делаются доступными по чтению всем пользователям.

```
#!/bin/sh
# выполнение операций над группой файлов
for FILE in $HOME/*.bash
do
    cp $FILE ${HOME}/scripts
    chmod a+r ${HOME}/scripts/${FILE}
done
```

Язык программирования оболочки содержит операторы, нарушающие нормальное выполнение цикла. Эти операторы имеют названия – *break* и *continue*. Они работают в точности так же, как и их аналоги в языке C.

Цикл *select* позволяет создавать удобное меню. Он полезен, когда необходимо, чтобы пользователь выбрал один элемент из предлагаемого списка. Оператор *select* имеет такой же вид, как и оператор *for*, за исключением ключевого слова.

При выполнении данного оператора цикла все элементы из списка высвечиваются на экране вместе с их порядковыми номерами, после чего появляется специальное приглашение для ввода. Обычно оно имеет вид *#?*. Введённый пользователем номер пункта меню записывается в переменную *REPLY*. Если *\$REPLY* содержит номер пункта меню, то в переменную *имя* заносится значение соответствующего элемента из списка. В противном случае список будет выдан заново.

После того как пользователем будет сделан допустимый выбор, выполняются команды из *список*, после чего выполнение цикла повторяется с самого начала (высветится приглашение для ввода и т. д.). Для повторного отображения меню в ответ на приглашение ввода следует нажать клавишу *Enter*. Выход из цикла осуществляется теми же средствами, что и для *while* и *for*.

Ниже приводится пример использования оператора *select*. У пользователя запрашивается тип устройства «мышь», и в зависимости от сделанного выбора выполняются определённые действия. В рассматриваемом случае это просто выдача сообщения, подтверждающая сделанный выбор. В приведённом сценарии обратим внимание на вторую строчку. В большинстве случаев действующий по умолчанию вид приглашения не устраивает. Определяя переменную *PS3*, можно задать необходимый текст приглашения.

```
#!/bin/sh
# “конфигурирование” устройства «мышь»
PS3=”Выберите тип устройства «мышь»:”
```

```

select ITEM in Microsoft Logitech ps2 none
do
    case $ITEM in
        Microsoft) echo "действия по «мышь» Microsoft " ;;
        Logitech) echo "действия по «мышь» Logitech " ;;
        ps2) echo "действия по «мышь» ps2 " ;;
        none) echo "выбрано – нет «мышь» " ;;
    esac
break
done

```

Использование функций

В сценариях оболочки допустимо определять и использовать функции. Под функцией понимается поименованная группа команд. Во всех случаях, когда в сценарии присутствует повторяющийся код с возможно небольшими вариациями, необходимо рассмотреть возможность применения функции. Так же логически завершённую группу команд удобнее представить отдельной функцией, что позволяет упростить чтение и восприятие сценария. Определение функции имеет следующий синтаксис:

```

имя_функции ()
{
    список команд
}

```

Определение функции обязательно должно предшествовать её первому использованию. В отличие от языка C++ в языке оболочки *bash* отсутствуют средства предварительного объявления функции (аналоги прототипов функций). Вызов функции осуществляется путём указания её имени в качестве команды.

Ниже приведён пример сценария, в котором определяется функция *about*, предназначенная для выдачи информации о сценарии, и иллюстрируется её использование:

```

#!/bin/sh
# определение функции
about()
{
    echo пример сценария с функцией
    echo (C) Иванов И.И., 2021
}
about          # вызов функции
# ... продолжение сценария

```

Функции при вызове можно передавать аргументы и при своём завершении функция может возвращать результат выполнения. Функция ссылается к переданным параметрам через позиционные переменные - \$1, \$2 и так далее. Результат работы возвращается с помощью оператора *return*. Ниже приведён пример сценария, в котором иллюстрируется доступ функции к переданным в неё параметрам и вызов функции с передачей параметров.

```
#!/bin/sh
# функция с параметром
double_echo()
{
    if [ -n "$1" ]
    then
        echo $1
        echo $1
    fi
    return 0
}
double_echo параметр          # вызов функции с одним параметром
double_echo                  # вызов функции без параметров
# ... продолжение сценария
```

3. Практическая часть

Пример 1: Создать скрипт, выводящую на экран текущую дату и время, полное маршрутное имя текущего каталога, полное маршрутное имя домашнего каталога пользователя, регистрационное имя пользователя, собственное имя скрипта:

```
#!/bin/bash
# Пример1
# Вывод на экран текущей даты и времени
date
# Вывод на экран полного маршрутного имени текущего каталога
pwd
# Вывод на экран полного маршрутного имени домашнего каталога
# пользователя
echo $HOME
# Вывод на экран регистрационного имени пользователя
who am i
# Вывод на экран собственного имени скрипта
echo $0
# Конец скрипта Пример1
```

Пример 2: Создать скрипт, выводящую на экран все переданные параметры: текущую дату и время, полное маршрутное имя текущего каталога, полное маршрутное имя домашнего каталога пользователя, регистрационное имя пользователя, собственное имя процедуры, количество переданных в скрипт параметров:

```
#!/bin/bash
# Пример2
# Вывод на экран значений первого, второго, третьего и четвертого
# позиционных
# параметров, переданных процедуре при обращении, а также нулевого
# параметра,
# содержащего имя выполняемой процедуры
echo $1 $2 $3 $4 $0
# Конец скрипта Пример2
```

Пример 3: Создать скрипт, записывающую в некоторый файл текст, вводимый с клавиатуры при работе процедуры. Имя файла передается в скрипт в качестве параметра. В конце текста должна быть текущая дата:

```
#!/bin/bash
# Пример3
echo Введите текст
read TEMP1
echo $TEMP1 >$1
date >TEMP2
cat $1 TEMP2 >TEMP1
mv TEMP1 $1
# Конец скрипта Пример3
```

Пример 4: Создать скрипт, вводящую в начало файла _1 некоторый текст с клавиатуры, затем присоединяющего содержимое файла _2, затем снова вводящего текст с клавиатуры в конец файла _1. Имена файлов передаются в скрипт в качестве параметров:

```
#!/bin/bash
# Пример4
cat $1 >TEMP1
cat TEMP1 $2 >$1
# Конец скрипта Пример4
```

Пример 5: Создать скрипт, находящую в файле, имя которого передается в качестве параметра, все строки, содержащие вводимую с клавиатуры во время работы скрипта последовательность символов:

```
#!/bin/bash
# Пример5
# Ввод с клавиатуры заданной последовательности символов и запись ее
в переменную A
read A
# Поиск в заданном файле всех строк, содержащих заданную
последовательность
grep $A $1
# Конец скрипта Пример5
```

Пример 6: Создать скрипт, вводящую с клавиатуры два целых числа и выводящую на экран их сумму, произведение, частное и остаток от деления первого числа на второе:

```
#!/bin/bash
# Пример6
```

```

# Ввод с клавиатуры двух целых чисел и присваивание их переменным A
и B.
read A B
# Вычисление суммы введенных чисел и присваивание ее переменной SUM
SUM=`expr $A + $B`
# Вычисление произведения введенных чисел и присваивание результата
переменной PROD
PROD=`expr $A \* $B`
# Вычисление частного от деления первого введенного числа на второе
и присваивание
# результата переменной QUOT
QUOT=`expr $A / $B`
# Вычисление остатка от деления первого введенного числа на второе и
присваивание
# результата переменной REMA
REMA=`expr $A % $B`

# Вывод на экран полученных результатов
echo Сумма введенных чисел равна
echo $SUM
echo Произведение введенных чисел равно
echo $PROD
echo Частное от деления первого числа на второе равно
echo $QUOT
echo Остаток от деления первого числа на второе равно
echo $REMA
# Конец скрипта Пример6

```

Пример 7: Создать скрипт, которая выводит на экран постранично содержимое одного из двух символьных файлов, имена которых передаются процедуре в качестве параметров. Выбрать файл с меньшим числом строк (код завершения 000). Если заданные файлы имеют одинаковое число строк, то вывести на экран содержимое обоих файлов (код завершения 111):

```

#!/bin/bash
# Пример7

# Запись в переменную TEMP1 числа строк в файле $1
TEMP1=`wc -l $1 | cut -f1 -d " "`
# Запись в переменную TEMP2 числа строк в файле $2
TEMP2=`wc -l $2 | cut -f1 -d " "`
# Выбор файла с меньшим числом строк

```

```

    if test $TEMP1 -lt $TEMP2
    then
# Вывод на экран содержимого файла $1
        more $1
# Выход из процедуры
        exit 000
    fi
if test $TEMP2 -lt $TEMP1
    then
        test
# Вывод на экран содержимого файла $2
        more $2
# Выход из процедуры
        exit 000
    fi
# Проверка равенства числа строк в файлах $1 и $2
    if test $TEMP1 -eq $TEMP2
    then
        test
# Вывод на экран содержимого файлов $1 и $2
        more $1 $2
# Выход из процедуры
        exit 111
    fi

# Конец скрипта Пример7

```

Пример 8: Создать скрипт, которая:

- среди пользователей, работающих в данный момент времени в системе, находит всех пользователей, имена которых содержатся во вводимой в процедуру символьной строке;
- выводит на экран найденные имена пользователей:

```

#!/bin/bash
# Пример8

# Формирование в файле file списка работающих пользователей
who >file
# Ввод заданных имен пользователей
echo "input string:"
read temp

```

```

# Присваивание позиционным параметрам заданных в строке #имен
пользователей
set $temp
# Поиск в файле file заданных имен и вывод их на экран
while [ "$1" ]
do
# Поиск в файле file очередного имени
    grep $1 file
    shift
done

# Конец скрипта Пример8

```

Пример 9: Создать бесконечный скрипт, который периодически, каждые 15 с, опрашивает систему и выводит на экран сведения о каждом вновь вошедшем в систему пользователе:

```

#!/bin/bash
# Пример9

# Сохранение сведений о пользователях, работающих в системе
# в момент запуска процедуры на выполнение
who >file1
# Сохранение списка имен работающих пользователей
cut -f1 -d ' ' file1 >file3
# Бесконечный цикл
while true
do
    # Пауза 15 с
    sleep 15
    # Сохранение сведений о работающих пользователях через 15 с
    who >file2
    # Сохранение списка имен работающих пользователей
    cut -f1 -d ' ' file2 >file4
    # Поиск каждого нового пользователя в списке старых
    # пользователей и вывод на экран
    # сведений о каждом вновь вошедшем в систему пользователе
    for newuser in file4
    do
        if grep $newuser file3
        then

```

```

                                continue
                        else
                                grep $newuser file2
                        fi
                done
                # Обновление сведений о работающих пользователях
                cp file4 file3
done

# Конец скрипта Пример9

```

Пример 10: Создать бесконечный скрипт, который осуществляет просмотр подкаталогов в текущем каталоге и выполнение команд, вводимых с терминала, до получения сигнала конец_файла или прерывание. (Сигнал прерывание действует только по окончании выполнения команды):

```

#!/bin/bash
# Пример10

d=`pwd`
for i in *
do
    if test -d $d/$i
    then
        cd $d/$i
        while echo $i:
            trap exit 2
            read x
        do trap : 2; eval $x; done
    fi
done
# Встроенная команда read x считывает строки из файла стандартного
ввода и
# присваивает результат их выполнения переменной x.
# Сигналы конец_файла или прерывание прекращают выполнение этой
команды с кодом
# завершения, не равным нулю.

# Конец скрипта Пример10

```

4. Задание

При старте сценарий необходимо выдавать информацию об авторе в виде:
(C) имя фамилия группа

Вариант 1

Написать shell-процедуру, которая:

- вводит передаваемое в качестве 1-го параметра количество символьных строк;
- в каждой введенной строке ищет подстроку, передаваемую в качестве второго параметра;
- заменяет каждую найденную подстроку на строку, передаваемую в качестве третьего параметра;
- выводит на экран каждую введенную строку и соответствующую ей новую строку.

Вариант 2

Написать shell-процедуру, которая:

- вводит 2 символьные строки;
- в каждой введенной строке ищет подстроку, передаваемую в качестве параметра; заменяет каждую найденную подстроку на пробел;
- образует из полученных строк третью строку так, чтобы в ней чередовались слова из первой и второй строк;
- выводит на экран введенные строки и новую строку.

Вариант 3

Написать shell-процедуру, которая:

- вводит символьную строку;
- во введенной строке ищет подстроку, передаваемую в качестве первого параметра;
- вставляет после каждой найденной подстроки символ, передаваемый в качестве второго параметра;
- удаляет из полученной строки символ, передаваемый в качестве третьего параметра; выводит на экран введенную и новую строку.

Вариант 4

Написать shell-процедуру, которая:

- вводит символьную строку;
- проверяет введенную строку на совпадение со строкой, переданной в качестве 1-го параметра;
- если строки совпадают, то выдает на экран приглашение повторить ввод;

- если не совпадают, то сравнивает длину введенной строки с длиной 2-го параметра, и, в случае их равенства, выводит на экран введенную строку в обратном порядке составляющих ее символов.

Вариант 5

Написать shell-процедуру, которая:

- проверяет введенную строку на совпадение со строками, содержащимися в файле, имя которого передается в качестве 1-го параметра;
- для всех найденных совпадений заменяет соответствующие строки в файле на строку, переданную в качестве 2-го параметра;
- выводит на экран старое и новое содержимое файла, а также число найденных совпадений.

Вариант 6

Написать shell-процедуру, которая:

- вводит символьную строку, содержащую маршрутное имя некоторого файла; проверяет введенное маршрутное имя, если оно начинается с символа /, на совпадение его первой части с маршрутным именем домашнего каталога пользователя;
- если введенное маршрутное имя содержит маршрутное имя домашнего каталога или является относительным, то проверяет существование указанного файла,
- в противном случае выводит на экран сообщение об ошибке; если файл существует, то выводит на экран его содержимое;
- если файл не существует, то создает его и записывает в него строку, передаваемую в качестве параметра.

Вариант 7

Написать shell-процедуру, которая:

- вводит символьную строку, содержащую имя некоторого файла; проверяет наличие файла в домашнем каталоге или в одном из подкаталогов;
- если файл существует, то выводит на экран его содержимое;
- если файл не существует, то создает его и записывает в него с консоли некоторый текст;
- устанавливает для файла права доступа, передаваемые в качестве параметра.

Вариант 8

Написать shell-процедуру, которая:

- вводит символьную строку, содержащую текст некоторого сообщения; проверяет наличие в своем почтовом ящике такого же сообщения;
- если в почтовом ящике имеется введенное сообщение, то выводит его на экран и посылает на терминалы всем пользователям, в данный момент работающим в системе из числа тех, чьи имена передаются в качестве параметров;

- всем остальным пользователям, чьи имена передаются в качестве параметров, рассылает введенное сообщение по почте.

Вариант 9

Написать shell-процедуру, которая:

- вводит символьную строку, содержащую текст некоторого сообщения;
- проверяет регистрацию в системе пользователей, чьи имена переданы вторым и последующими параметрами;
- всем пользователям, чьи имена передаются в качестве второго и следующих параметров и работающим в системе в течение заданного первым параметром времени, рассылает введенное сообщение по почте;
- всем остальным пользователям, работающим в данный момент в системе, рассылает прямые сообщения, содержащие введенную символьную строку.

Вариант 0

Написать shell-процедуру, которая:

- проверяет свой почтовый ящик на наличие в нем сообщений;
- находит в почтовом ящике одинаковые по тексту сообщения;
- всем пользователям, приславшим более одного сообщения с одинаковым текстом, рассылает по почте сообщения, текст которого содержится в файле, имя которого передается в качестве параметра.